Markdown Studio — Feature Demo

Open this file and run **Markdown Studio: Preview** (`Cmd+Shift+P`).

- Markdown Studio — Feature Demo
  - 1. Markdown Rendering
    - 日本語テスト
    - Task Lists
    - Footnotes
    - Emoji
    - Math (KaTeX)
    - Definition Lists
    - Superscript / Subscript
  - 2. Mermaid Diagrams
    - Markdown Studio Architecture
    - Extension Activation Flow
  - 3. PlantUML Diagrams
    - Extension Component Diagram
    - Document Processing Sequence
  - 4. Inline SVG (Sanitized)
  - 5. Syntax Highlighting
  - 6. Images and Security
    - ✅ Local image (relative path — works)
    - ✅ Local image (absolute path — works)
    - ❌ External image (blocked by policy)
    - ❌ External link (blocked by policy)
    - How to allow external resources
    - Security Summary
  - 7. Theme Adaptability
  - 8. Diagram Type Catalog
  - 9. Custom CSS
    - Bundled Themes
    - Setup
    - How It Works
  - 10. PDF Export

---

1. Markdown Rendering

Bold, *italic*, ~~strikethrough~~, `inline code`

> Blockquotes work too.

Feature	Status	Remark
Markdown	✅	A

Feature	Status	Remark
Mermaid	✅	B
PlantUML	✅	C
SVG	✅	D
PDF Export	✅	E

1. Ordered list item
2. Another item
3. Third item

- Unordered item
- Another item

- Item

日本語テスト

- 日本語あいうえお
  - 2byte文字テスト

Task Lists

- [x] Markdown rendering
- [x] Mermaid diagrams
- [x] PlantUML diagrams
- [x] LaTeX math
- [x] Footnotes
- [ ] no checked

Footnotes

This sentence has a footnote[^1] and another one[^2].

[^1]: This is the first footnote content. [^2]: This is the second footnote with **bold** text.

Emoji

😄🚀👍⚠️❤️⭐🔥☕

Math (KaTeX)

Inline math: $E = mc^2$ and $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$

Display math:

$$ \int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} $$

$$ f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} $$

Definition Lists

Markdown : A lightweight markup language for creating formatted text.

PlantUML : A tool for creating UML diagrams from plain text. : Uses Smetana layout engine (no Graphviz needed).

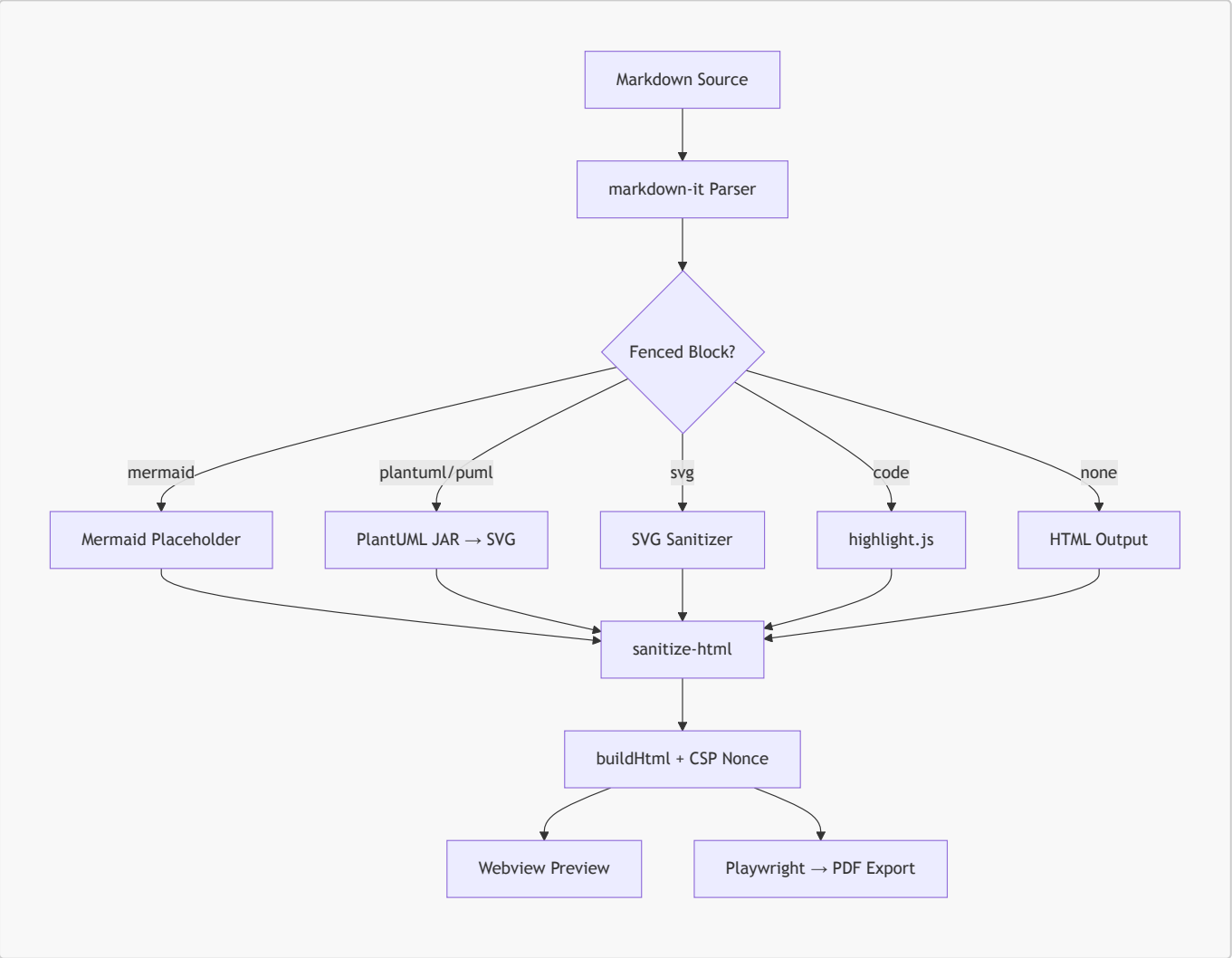
Superscript / Subscript

- Water: H~2~O
- Einstein: E = mc^2^
- 19th century

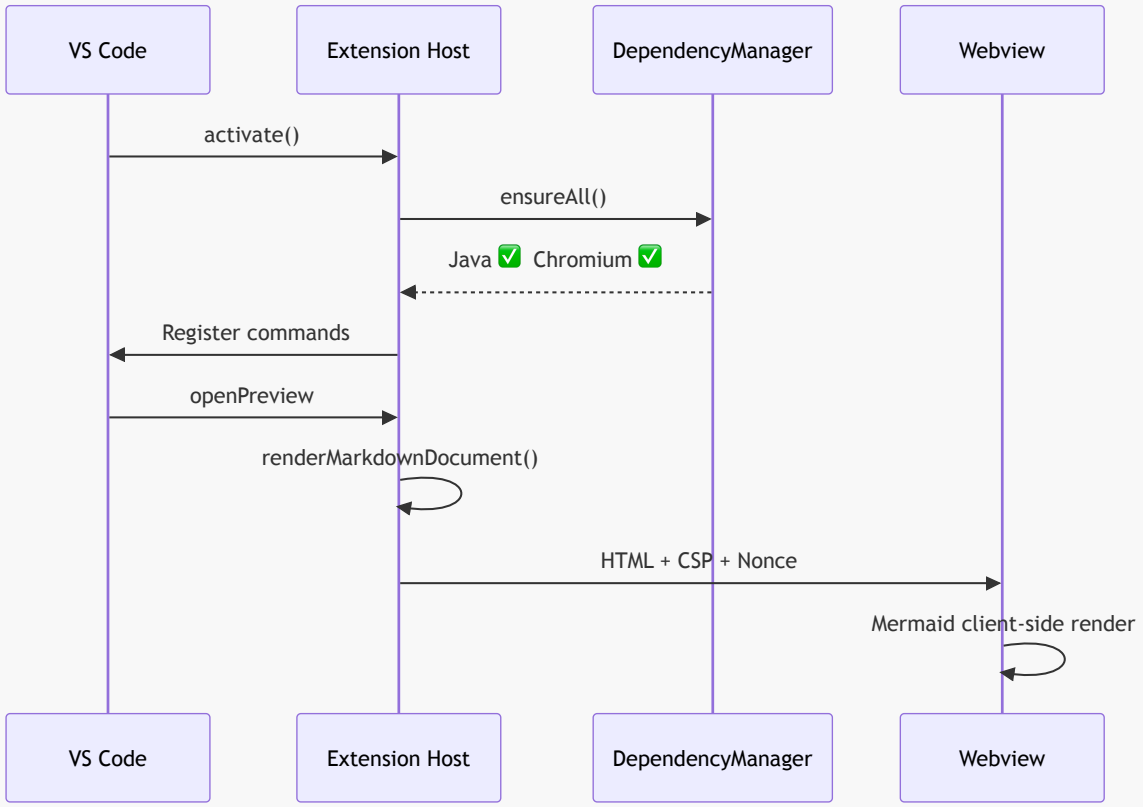
---

2. Mermaid Diagrams

Markdown Studio Architecture



Extension Activation Flow



---

3. PlantUML Diagrams

Rendered locally via bundled JAR. No remote server.

Extension Component Diagram

```
@startuml
skinparam componentStyle rectangle
skinparam defaultFontSize 14

package "Markdown Studio" {
  [Extension Host] as ext
  [Markdown Parser] as parser
  [Mermaid Renderer] as mermaid
  [PlantUML Renderer] as plantuml
  [SVG Sanitizer] as svg
  [HTML Builder] as html
  [PDF Exporter] as pdf
  [Webview Preview] as preview
}

package "External Dependencies" {
  [Amazon Corretto JDK] as java
  [Playwright Chromium] as chromium
  [PlantUML JAR] as jar
}
```

```
ext --> parser
parser --> mermaid
parser --> plantuml
parser --> svg
parser --> html
html --> preview
html --> pdf

plantuml --> jar
jar --> java
pdf --> chromium
@enduml
```

Document Processing Sequence

```
@startuml
skinparam defaultFontSize 14

actor Developer
participant "VS Code" as vsc
participant "Extension" as ext
participant "PlantUML JAR" as jar
participant "Webview" as wv

Developer -> vsc : Open .md file
Developer -> vsc : Cmd+Shift+P → Preview
vsc -> ext : openPreview command
ext -> ext : scanFencedBlocks()
ext -> ext : renderMermaidBlock() → placeholder
ext -> jar : java -jar plantuml.jar -tsvg
jar --> ext : SVG output
ext -> ext : sanitizeSvg()
ext -> ext : sanitizeHtmlOutput()
ext -> ext : buildHtml() + CSP nonce
ext -> wv : Set webview HTML
wv -> wv : Mermaid client-side render
wv --> Developer : Live preview
@enduml
```

---

4. Inline SVG (Sanitized)

Dangerous elements (`<script>`, `<foreignObject>`, event handlers) are stripped automatically.

```svg
<svg viewBox="0 0 600 200" xmlns="http://www.w3.org/2000/svg" width="100%">
  <!-- Parse stage -->
  <rect x="30" y="20" width="160" height="70" rx="12" fill="#4CAF50"/>
  <text x="110" y="55" text-anchor="middle" fill="white" font-size="15"
```

```svg
font-weight="bold">Parse</text>
  <text x="110" y="75" text-anchor="middle" fill="white" font-
size="11">markdown-it + scanner</text>

  <!-- Arrow -->
  <polygon points="195,55 215,47 215,63" fill="#888"/>

  <!-- Render stage -->
  <rect x="220" y="20" width="160" height="70" rx="12" fill="#2196F3"/>
  <text x="300" y="55" text-anchor="middle" fill="white" font-size="15"
font-weight="bold">Render</text>
  <text x="300" y="75" text-anchor="middle" fill="white" font-
size="11">Mermaid · PlantUML · SVG</text>

  <!-- Arrow -->
  <polygon points="385,55 405,47 405,63" fill="#888"/>

  <!-- Export stage -->
  <rect x="410" y="20" width="160" height="70" rx="12" fill="#FF9800"/>
  <text x="490" y="55" text-anchor="middle" fill="white" font-size="15"
font-weight="bold">Export</text>
  <text x="490" y="75" text-anchor="middle" fill="white" font-
size="11">Playwright → PDF</text>

  <!-- Security icons -->
  <circle cx="110" cy="140" r="25" fill="#4CAF50" opacity="0.2"
stroke="#4CAF50" stroke-width="2"/>
  <text x="110" y="145" text-anchor="middle" fill="#4CAF50" font-size="16">
🔒 </text>
  <text x="110" y="175" text-anchor="middle" fill="#4CAF50" font-
size="9">CSP + Nonce</text>

  <circle cx="300" cy="140" r="25" fill="#2196F3" opacity="0.2"
stroke="#2196F3" stroke-width="2"/>
  <text x="300" y="145" text-anchor="middle" fill="#2196F3" font-size="16">
🛡️ </text>
  <text x="300" y="175" text-anchor="middle" fill="#2196F3" font-
size="9">HTML Sanitize</text>

  <circle cx="490" cy="140" r="25" fill="#FF9800" opacity="0.2"
stroke="#FF9800" stroke-width="2"/>
  <text x="490" y="145" text-anchor="middle" fill="#FF9800" font-size="16">
💻 </text>
  <text x="490" y="175" text-anchor="middle" fill="#FF9800" font-
size="9">Local-First</text>

  <!-- Connecting dashed lines -->
  <line x1="110" y1="90" x2="110" y2="115" stroke="#4CAF50" stroke-
width="2" stroke-dasharray="4"/>
  <line x1="300" y1="90" x2="300" y2="115" stroke="#2196F3" stroke-
width="2" stroke-dasharray="4"/>
  <line x1="490" y1="90" x2="490" y2="115" stroke="#FF9800" stroke-
```

```
width="2" stroke-dasharray="4"/>
</svg>
```

---

5. Syntax Highlighting

```
import * as vscode from 'vscode';

export function activate(context: vscode.ExtensionContext): void {
  const depManager = new DependencyManager();
  const status = await depManager.ensureAll(context);

  context.subscriptions.push(
    vscode.commands.registerCommand('markdownStudio.openPreview', async ()
=> {
      await openPreviewCommand(context);
    }),
    vscode.commands.registerCommand('markdownStudio.exportPdf', async () =>
{
      await exportPdfCommand(context);
    })
  );
}
```

```
{
  "markdownStudio.plantuml.mode": "bundled-jar",
  "markdownStudio.java.path": "java",
  "markdownStudio.export.pageFormat": "A4",
  "markdownStudio.security.blockExternalLinks": true
}
```

```
# PlantUML rendering is also useful for Python projects
from dataclasses import dataclass

@dataclass
class DiagramConfig:
    mode: str = "bundled-jar"
    java_path: str = "java"
    timeout_ms: int = 15000
```

```
fmt.Println("Hello, World!")
```

```
plain text without language
```

```rust
fn main() {
    println!("Hello from Rust");
}
```

```sql
SELECT id, name FROM users WHERE active = true;
```

```dockerfile
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
```

---

6. Images and Security

Markdown Studio is local-first. The default security policy (`blockExternalLinks = true`) blocks external resources while allowing local files.

✅ Local image (relative path — works)

- local svg

- icon.png

- icon.svg

Relative paths from the Markdown file are resolved and displayed correctly.

✅ Local image (absolute path — works)

Images with absolute file paths also work in the preview.

❌ External image (blocked by policy)

Remote image

External URLs (`https://...`) are blocked by default. The image above shows a policy notice instead.

❌ External link (blocked by policy)

External link — blocked by default ✋

External links are replaced with a blocked notice in the preview.

How to allow external resources

Set `markdownStudio.security.blockExternalLinks` to `false` in VS Code settings to allow external images and links. This is not recommended for corporate/secure environments.

Security Summary

Resource Type	Default	Configurable
Local images (relative path)	✅ Allowed	—
Local images (absolute path)	✅ Allowed	—
External images (https://)	❌ Blocked	`blockExternalLinks: false`
External links (https://)	❌ Blocked	`blockExternalLinks: false`
Inline SVG	✅ Allowed	—
Mermaid diagrams	✅ Allowed	—
PlantUML diagrams	✅ Allowed	—

---

7. Theme Adaptability

Switch between **light** and **dark** mode in VS Code (`Cmd+K Cmd+T`) to see how the preview adapts:

- Mermaid diagrams automatically switch between light and dark themes

- SVG elements use colors chosen for visibility in both themes
- Code blocks use theme-aware syntax highlighting
- PlantUML output receives CSS overrides for dark mode

---

8. Diagram Type Catalog

All diagram types below are verified to render correctly with the bundled PlantUML + Smetana engine (no Graphviz required).

▶ Class Diagram
▶ Activity Diagram
▶ State Diagram
▶ Use Case Diagram
▶ Timing Diagram
▶ Mind Map
▶ Gantt Chart
▶ Object Diagram
▶ Mermaid: Pie Chart
▶ Mermaid: Git Graph
▶ Mermaid: XY Chart
▶ SVG: Basic Shapes
▶ SVG: Lines and Paths
▶ SVG: Text and Styling
▶ SVG: Dashboard Layout

---

9. Custom CSS

Markdown Studio supports built-in themes and inline CSS customization.

Bundled Themes

Select a theme from the settings dropdown (`markdownStudio.style.theme`):

Theme	Description
`default`	No extra styling — uses the selected preset only
`modern`	Indigo accents, soft shadows, refined typography
`markdown-pdf`	Classic look matching the popular Markdown PDF extension
`minimal`	Bare-bones, clean starting point

All themes include dark mode support and print-optimized styles.

Setup

```
// Pick a theme from the dropdown
"markdownStudio.style.theme": "modern"
```

```
// Optionally add your own CSS on top
"markdownStudio.style.customCss": "h1 { color: navy; } blockquote { border-left: 3px solid orange; }"
```

How It Works

- Theme CSS is injected **after** the preset styles, so theme rules take priority
- `customCss` is applied **after** the theme, so your inline rules override everything
- The same CSS applies to both preview and PDF export for consistent output
- Copy any bundled theme and customize it to create your own

---

10. PDF Export

This entire document can be exported to PDF:

1. Open this file in VS Code
2. `Cmd+Shift+P` → **Markdown Studio: Export PDF**
3. A `demo.pdf` will be generated next to this file

The PDF uses the same HTML pipeline as the preview.

Markdown Studio v0.7.0